

MODUL I

PEMROGRAMAN DASAR

1.1 TUJUAN PRAKTIKUM

Tujuan Praktikum pada Modul I Pemrograman Dasar adalah:

1. Mahasiswa mengetahui alur pembuatan algoritma, pseudocode, dan *flowchart*.
2. Mahasiswa mengetahui dasar-dasar penulisan program komputer.
3. Mahasiswa mampu memahami struktur bahasa C++.
4. Mahasiswa mengetahui dan mampu menggunakan tipe data primitif dalam pembuatan program.
5. Mahasiswa mampu memahami typecasting variabel.

1.2 DASAR TEORI

1.2.1 Algoritma

Algoritma adalah sebuah metode yang digunakan untuk menyelesaikan sebuah permasalahan dengan langkah-langkah yang sistematis guna mendapatkan sebuah hasil yang lebih baik [1]. Algoritma melibatkan dua hal, yaitu objek dan operasi. Objek adalah sesuatu yang akan diubah/dimanipulasi oleh algoritma, dan operasi adalah segala sesuatu yang dikerjakan terhadap objek tersebut [2]. Perhatikan contoh algoritma membuat telur ceplok berikut.

1. Menyiapkan bahan-bahan (minyak goreng, telur, garam).
2. Panaskan minyak di atas wajan.
3. Pecahkan telur.
4. Masukkan telur ke dalam wajan dengan minyak yang sudah panas.
5. Beri garam.
6. Balik telur saat sudah matang.
7. Angkat telur setelah matang sempurna.
8. Telur siap dimakan.

1.2.2 Pseudocode

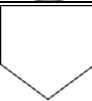
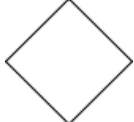
Berbeda dengan algoritma deskriptif, algoritma *pseudocode* bukanlah sebuah kode program. Algoritma ini merupakan sebuah deskripsi algoritma dengan struktur yang lebih sederhana dari beberapa bahasa pemrograman agar lebih mudah dimengerti oleh manusia. *Pseudocode* memiliki beberapa struktur dalam menuliskannya. Deklarasi program berfungsi untuk menuliskan nama program yang akan digunakan.

Kamus digunakan untuk mendeklarasikan variabel apa saja yang akan digunakan. Selanjutnya, terdapat deskripsi algoritma yang digunakan untuk mendeskripsikan jalannya algoritma dalam sebuah program [1].

1.2.3 Flowchart

Flowchart merupakan suatu bagan dengan simbol-simbol khusus yang bertujuan untuk memberikan gambaran terkait urutan proses dan hubungan antara suatu proses (intruksi) dengan proses lainnya dalam suatu algoritma. Dalam perancangan *flowchart* sebenarnya tidak ada patokan mutlak yang harus diikuti, hal yang penting adalah kesesuaian penggunaan simbol-simbol. Secara garis besar, terdapat 3 bagian utama dalam sebuah *flowchart* yaitu *input*, proses, dan *output* [3]. Adapun simbol-simbol *flowchart* dapat dilihat pada **Tabel 1.1**

Tabel 1.1 Simbol *Flowchart*

No.	Simbol	Nama
1.		Simbol <i>Flow Direction</i>
2.		Simbol <i>Terminator</i>
3.		Simbol <i>On Page Connector</i>
4.		Simbol <i>Off Page Connector</i>
5.		Simbol <i>Process</i>
6.		Simbol <i>Decision</i>
7.		Simbol <i>Input-Output</i>
8.		Simbol <i>Predefine Process</i>

Berdasarkan **Tabel 1.1** di atas, dapat diketahui bahwa simbol *flow direction* digunakan untuk menghubungkan antara simbol yang satu dengan simbol yang lainnya. Simbol *terminator* merupakan simbol permulaan (*start*) dan pengakhiran (*stop*) dari suatu kegiatan. Simbol *on page connector* menghubungkan proses dalam lembar/halaman yang sama (diberi label). Biasanya digunakan saat kesulitan menghubungkan 2 buah proses secara langsung. Simbol *off page connector* sama seperti simbol konektor lingkaran, namun simbol ini digunakan untuk menghubungkan proses dalam halaman yang berbeda (diberi label). Simbol *process* menunjukkan pengolahan/pemrosesan yang dilakukan oleh komputer.

Simbol *decision* berfungsi untuk penyeleksian data yang memberikan pilihan. Simbol *input-output* menyatakan proses *input* dan *output* data. Simbol *predefine proces* digunakan untuk pelaksanaan suatu bagian (sub-program)/prosedur [3].

1.2.4 Tipe Data

Tipe data merupakan jenis data yang digunakan untuk merepresentasikan jenis/tipe nilai yang dapat ditampung oleh sebuah variabel [3]. Berikut merupakan jenis-jenis tipe data dasar (*primitive data type*).

1. *Integer*

Yakni sebuah bentuk tipe data berupa bilangan bulat atau numerik bukan angka pecahan. Tipe ini biasanya merupakan bilangan dari -707,0 hingga 707. Pada tipe ini, bilangan bulat yang tersimpan memiliki panjang 16 hingga 65.536-bit [1].

2. *Float*

Yakni tipe data untuk menyimpan angka yang memiliki komponen berupa pecahan. Tipe data ini biasanya ditulis dalam bentuk desimal. Untuk penyimpanannya, tipe data *float* mampu disimpan dalam area 32-bit [1].

3. *Boolean*

Yakni tipe data untuk memilih jawaban benar (*true*) atau salah (*false*) berupa bilangan 1 atau 0. Selain itu, bentuk tipe data ini hanya bisa menggambarkan 2 nilai yang dipertimbangkan layaknya sebuah hubungan angka yang lebih kecil maupun angka yang lebih besar [1].

4. *Char*

Salah satu tipe data yang paling banyak digunakan selain integer. Tipe ini digunakan untuk menyimpan huruf, angka, tanda baca, maupun simbol. Variabel yang didefinisikan menampung tipe data ini membutuhkan 1-bit [1].

1.2.5 Operator

Operator adalah dasar dari bahasa pemrograman apapun. Dengan demikian fungsionalitas bahasa pemrograman C++ tidak lengkap atau tidak dapat berjalan tanpa menggunakan operator. Dalam membuat program, tentunya akan ada banyak data yang diolah sehingga menghasilkan *output*. Pengolahan data tersebut tentu membutuhkan operator [3].

Maka dari itu, kita dapat mendefinisikan operator sebagai simbol yang membantu kita melakukan perhitungan matematis dan logis tertentu pada operand. Dengan kata lain, kita dapat mengatakan bahwa operator mengoperasikan operand [3].

Operator adalah simbol atau karakter khusus yang digunakan untuk melakukan suatu operasi membantu *compiler* untuk memanipulasi secara matematis atau logis pada data yang diberikan. Bahasa pemrograman C++ menyediakan banyak operator yang sangat berguna untuk melakukan operasi. Terdapat beberapa jenis operator dalam pemrograman yaitu operator aritmatika, operator perbandingan, operator logika, operator *assignment*, operator *increment* dan *decrement* serta operator *bitwise* [3].

1.2.6 Typecasting

Typecasting adalah proses mengubah tipe data dari satu jenis ke jenis lain. Dalam pemrograman, *typecasting* dipakai agar suatu nilai dapat diolah sesuai tipe yang dibutuhkan, misalnya mengubah bilangan bulat menjadi pecahan agar pembagian menghasilkan nilai decimal [5].

1.2.7 Cin dan Cout

Fungsi *cin* (*console in*) ialah mengambil sebuah masukan. Operasinya membutuhkan variabel untuk menyimpan data. *cout* adalah sebuah objek dalam pemrograman C++ yang berfungsi untuk menampilkan semua jenis data (numerik dan karakter) di layar *console* berupa nilai dalam bentuk variabel maupun konstanta [1].

1.3 PERMASALAHAN

Fathan dan Bina sedang bertugas sebagai kasir di Toko Sembako Jaya Abadi. Pelanggan tetap mereka, Rendy, datang membeli beras, telur, dan minyak goreng. Tugasmu adalah membuat program kasir otomatis untuk menghitung total belanja, pajak, dan rincian uang kembalian dalam berbagai pecahan

1. *Input Harga*: Program menerima *input* harga untuk Beras, Telur, dan Minyak Goreng.

```
=====
|          TOKO SEMBAKO JAYA ABADI          |
=====
[Bina]: Than, masukan harga barangnya Rendy di program ini!
Harga Beras (Rp)           : 50000
Harga Telur (Rp)           : 30000
Harga Minyak Goreng (Rp): 20000
=====
```

Gambar 1.1 Contoh *Output Input Harga*

2. *Total Belanja, Pajak dan Menerima Uang dari Pelanggan*: Program menjumlahkan seluruh harga dan menambahkan pajak (PPN) sebesar 25%, dan menerima *input* nominal uang yang dibayar pelanggan.

```
Total Belanja (+ PPN 25%)      : Rp 125000
[Fathan]: Ren, totalnya segitu. Uang lu berapa?
[Rendy]: Hmmmmmmmm, pajaknya gede juga ya.
Nih than, my uang gweh segini
Nominal Uang Rendy (Rp)       : 200000
```

Gambar 1.2 Contoh *Output Total Belanja*

3. *Rincian Kembalian dan Struk*: Program menghitung total kembalian dan memecahnya ke dalam satuan uang kertas (Rp100rb - Rp1rb) dan koin (Rp1rb - Rp100) dari nilai terbesar ke terkecil, dan menampilkan rincian pecahan serta sisa uang yang tidak dapat dipecah (donasi).

```
=====
|          STRUK KEMBALIAN          |
=====
[Bina]: Nih, Than. Rincian kembaliannya:
Total Kembalian : Rp 75000
=====
- Rp 100.000 : 0 lembar
- Rp 50.000  : 1 lembar
- Rp 20.000  : 1 lembar
- Rp 10.000  : 0 lembar
- Rp 5.000   : 1 lembar
- Rp 1.000   : 0 lembar
- Rp 1.000   : 0 koin
- Rp 500     : 0 koin
- Rp 200     : 0 koin
- Rp 100     : 0 koin
=====
[Fathan]: Aman! Sisa receh tak terpecahnya: Rp 0
Ini mau di Donasikan ga Ren?
[Rendy]: GAS
=====
```

Gambar 1.3 Contoh *Output Rincian Kembalian dan Struk*

1.4 HASIL PERCOBAAN

1. Algoritma

- a. Mulai.
- b. Deklarasi variabel beras, telur, minyakGoreng, totalBelanja, uangRendy, uangKembalian, lembarKoin dengan tipe data integer.
- c. Tampilkan *header* "TOKO SEMBAKO JAYA ABADI".
- d. *Input* harga beras, telur, minyakGoreng.
- e. Hitung $\text{totalBelanja} = \text{beras} + \text{telur} + \text{minyakGoreng}$.
- f. Hitung $\text{totalBelanja} = \text{totalBelanja} + (\text{totalBelanja} * 25/100)$.
- g. Tampilkan totalBelanja setelah PPN.
- h. *Input* uangRendy.
- i. Hitung $\text{uangKembalian} = \text{uangRendy} - \text{totalBelanja}$.
- j. Tampilkan uangKembalian
- k. Pecah kembalian ke pecahan uang dari terbesar ke terkecil dengan hitung $\text{lembarKoin} = \text{uangKembalian} / \text{pecahan uang}$, sisa $\text{uangKembalian} = \text{uangKembalian} \% \text{pecahan uang}$.
- l. Tampilkan sisa receh yang tidak terpecah = sisa kembalian
- m. Berhenti.

2. Pseudocode

```

Step 1:  MULAI
Step 2:  DEKLARASI
         beras, telur, minyakGoreng
Step 3:  PRINT "=====
Step 4:  PRINT "|          TOKO SEMBAKO JAYA ABADI          |"
Step 5:  PRINT "=====
Step 6:  PRINT "[Bina]: Than, masukin harga barangnya Rendy di
         program ini!"
Step 7:  PRINT
         "Harga Beras (Rp) : "
Step 8:  INPUT beras
Step 9:  PRINT
         "Harga telur (Rp) : "
Step 10: INPUT telur
Step 11: PRINT
         "Harga Minyak Goreng (Rp) : "
Step 12: INPUT minyakGoreng
Step 13: DEKLARASI
         totalBelanja, uangRendy
Step 14: SET
         totalBelanja ← beras + telur + minyakGoreng
         totalBelanja ← totalBelanja + (totalBelanja * 25 / 100)
Step 15: PRINT "Total Belanja (+ PPN 25%) : ", totalBelanja
Step 16: PRINT "[Fathan]: Ren, totalnya segitu. Uang lu berapa?
Step 17: PRINT "[Rendy] : Hmmmmmm, pajaknya gede juga ya.
    
```

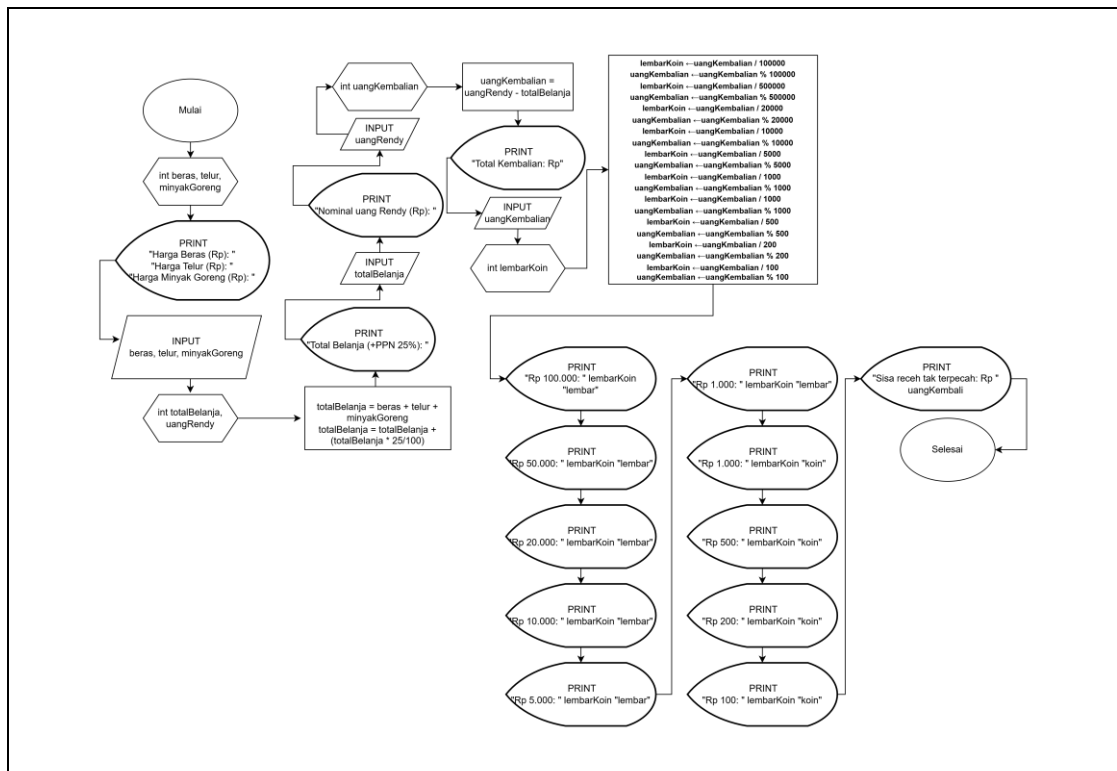
```

                                Nih than, my uang gweh segini"
Step 18:    PRINT "Nominal Uang Rendy (Rp) : "
Step 19:    INPUT uangRendy
Step 20:    DEKLARASI
            uangKembalian
Step 21:    SET uangKembalian = uangRendy - totalBelanja

Step 22:    PRINT "=====
Step 23:    PRINT "|                STUK KEMBALIAN                |"
Step 24:    PRINT "=====
Step 25:    PRINT "[Bina]: Nih, than. Rincian kembaliannya: "
Step 26:    PRINT "Total Kembalian: Rp ", uangKembalian
Step 27:    PRINT "-----"
Step 28:    DEKLARASI
            lembarKoin
Step 29:    SET
            lembarKoin ← uangKembalian / 100000
            uangKembalian ← uangKembalian MOD 100000
Step 30:    PRINT "- Rp 100.000 : ", lembarKoin, "lembar"
Step 31:    lembarKoin ← uangKembalian / 50000
            uangKembalian ← uangKembalian MOD 50000
Step 32:    PRINT "- Rp 50.000 : ", lembarKoin, "lembar"
Step 33:    lembarKoin ← uangKembalian / 20000
            uangKembalian ← uangKembalian MOD 20000
Step 34:    PRINT "- Rp 20.000 : ", lembarKoin, "lembar"
Step 35:    lembarKoin ← uangKembalian / 10000
            uangKembalian ← uangKembalian MOD 10000
Step 36:    PRINT "- Rp 10.000 : ", lembarKoin, "lembar"
Step 37:    lembarKoin ← uangKembalian / 5000
            uangKembalian ← uangKembalian MOD 5000
Step 38:    PRINT "- Rp 5.000 : ", lembarKoin, "lembar"
Step 39:    lembarKoin ← uangKembalian / 1000
            uangKembalian ← uangKembalian MOD 1000
Step 40:    PRINT "- Rp 1.000 : ", lembarKoin, "lembar"
Step 41:    lembarKoin ← uangKembalian / 1000
            uangKembalian ← uangKembalian MOD 1000
Step 42:    PRINT "- Rp 1.000 : ", lembarKoin, "koin"
Step 43:    lembarKoin ← uangKembalian / 500
            uangKembalian ← uangKembalian MOD 500
Step 44:    PRINT "- Rp 500 : ", lembarKoin, "koin"
Step 45:    lembarKoin ← uangKembalian / 200
            uangKembalian ← uangKembalian MOD 200
Step 46:    PRINT "- Rp 200 : ", lembarKoin, "koin"
Step 47:    lembarKoin ← uangKembalian / 100
            uangKembalian ← uangKembalian MOD 100
Step 48:    PRINT "- Rp 100 : ", lembarKoin, "koin"
Step 49:    PRINT "-----"
Step 50:    PRINT "[Fathan]: Aman! Sisa receh tak terpecahnya: Rp ",
            uangKembalian
Step 51:    PRINT "                Ini mau di Donasikan ga Ren? "
Step 52:    PRINT "[Rendy] : GAS"
Step 53:    PRINT "=====
Step 54:    SELESAI

```

3. Flowchart



Gambar 1.4 Flowchart Program Toko Sembako Jaya Abadi

4. Source Code

```
#include <iostream>
using namespace std;
int main(){
    int beras, telur, minyakGoreng; //Deklarasi beras, telur, minyak
    cout << "===== " << endl;
    cout << "|          TOKO SEMBAKO JAYA ABADI          |" << endl;
    cout << "===== " << endl <<
endl;
    cout << "[Bina]: Than, masukan harga barangnya Rendy di program ini!" <<
endl << endl;
    cout << "Harga Beras (Rp)          : "; cin >> beras;
    cout << "Harga Telur (Rp)          : "; cin >> telur;
    cout << "Harga Minyak Goreng (Rp) : "; cin >> minyakGoreng;
    //Hitung Total Belanja
    int totalBelanja = beras + telur + minyakGoreng; // Deklarasi Total dari
    beras, telur, minyak
    totalBelanja = totalBelanja + (totalBelanja * 25/100); //Deklarasi
    dan menghitung PPN
    int uangRendy;
    cout << "Total Belanja (+ PPN 25%) : " << totalBelanja << endl << endl;
    cout << "[Fathan]: Ren, totalnya segitu. Uang lu berapa?" << endl;
    cout << "[Rendy] : Hmmmmm, pajaknya gede juga ya." << endl;
    cout << "          Nih than, my uang gweh segini" << endl <<endl;
    cout << "Nominal Uang Rendy (Rp) : "; cin >> uangRendy;
```



```

cout<< endl;
//Hitung Uang Kembali
int uangKembalian = uangRendy - totalBelanja; //Deklarasi dan menghitung
uang kembalian
cout << "===== " << endl;
cout << "|                STRUK KEMBALIAN                |" << endl;
cout << "===== " << endl;
cout << "[Bina]: Nih, than. Rincian kembaliannya: " << endl;
cout << "Total Kembali: Rp " << uangKembalian << endl;
cout << "-----" << endl;
int lembarKoin = uangKembalian/100000;
    uangKembalian = uangKembalian % 100000;
cout << "- Rp 100.000 : " << lembarKoin << " lembar" << endl;
lembarKoin = uangKembalian/50000;
uangKembalian = uangKembalian % 50000;
cout << "- Rp 50.000 : " << lembarKoin << " lembar" << endl;
lembarKoin = uangKembalian/20000;
uangKembalian = uangKembalian % 20000;
cout << "- Rp 20.000 : " << lembarKoin << " lembar" << endl;
lembarKoin = uangKembalian/10000;
uangKembalian = uangKembalian % 10000;
cout << "- Rp 10.000 : " << lembarKoin << " lembar" << endl;
lembarKoin = uangKembalian/5000;
uangKembalian = uangKembalian % 5000;
cout << "- Rp 5.000 : " << lembarKoin << " lembar" << endl;
lembarKoin = uangKembalian/1000;
uangKembalian = uangKembalian % 1000;
cout << "- Rp 1.000 : " << lembarKoin << " lembar" << endl;
lembarKoin = uangKembalian/1000;
uangKembalian = uangKembalian % 1000;
cout << "- Rp 1.000 : " << lembarKoin << " koin" << endl;
lembarKoin = uangKembalian/500;
uangKembalian = uangKembalian % 500;
cout << "- Rp 500 : " << lembarKoin << " koin" << endl;
lembarKoin = uangKembalian/200;
uangKembalian = uangKembalian % 200;
cout << "- Rp 200 : " << lembarKoin << " koin" << endl;
lembarKoin = uangKembalian/100;
uangKembalian = uangKembalian % 100;
cout << "- Rp 100 : " << lembarKoin << " koin" << endl;
cout << "-----" << endl;
cout << "[Fathan]: Aman! Sisa receh tak terpecahnya: Rp " <<
uangKembalian << endl;
cout << "          Ini mau di Donasikan ga Ren? " << endl;
cout << "[Rendy] : GAS" << endl;
cout << "===== " << endl;
    return 0;
}

```

5. Hasil Program

```

=====
|          TOKO SEMBAKO JAYA ABADI          |
=====

[Bina]: Than, masukan harga barangnya Rendy di program ini!

Harga Beras (Rp)      : 50000
Harga Telur (Rp)      : 30000
Harga Minyak Goreng (Rp) : 20000
Total Belanja (+ PPN 25%) : 125000

[Fathan]: Ren, totalnya segitu. Uang lu berapa?
[Rendy] : Hhhhhhhh, pajaknya gede juga ya.
        Nih than, my uang gweh segini

Nominal Uang Rendy (Rp) : 200000

=====
|          STRUK KEMBALIAN          |
=====

[Bina]: Nih, than. Rincian kembaliannya:
Total Kembalian: Rp 75000
-----
- Rp 100.000 : 0 lembar
- Rp 50.000  : 1 lembar
- Rp 20.000  : 1 lembar
- Rp 10.000  : 0 lembar
- Rp 5.000   : 1 lembar
- Rp 1.000   : 0 lembar
- Rp 1.000   : 0 koin
- Rp 500     : 0 koin
- Rp 200     : 0 koin
- Rp 100     : 0 koin
-----

[Fathan]: Aman! Sisa receh tak terpecahnya: Rp 0
        Ini mau di Donasikan ga Ren?
[Rendy]  : GAS
=====

```

Gambar 1.5 Hasil Program Toko Sembako Jaya Abadi

Berdasarkan **Gambar 1.5** di atas, dapat diketahui bahwa program “TOKO SEMBAKO JAYA ABADI” dijalankan dengan memasukkan harga beras sebesar Rp 50.000, harga telur Rp 30.000, dan harga minyak goreng Rp 20.000. Program kemudian melakukan kalkulasi total belanja menggunakan operasi penjumlahan, menghasilkan subtotal Rp 100.000. Nilai tersebut selanjutnya dikenakan PPN 25% melalui operasi perkalian dan pembagian integer, sehingga total akhir yang harus dibayarkan menjadi Rp 125.000. Setelah total belanja ditampilkan, program meminta nominal uang yang dibawa Rendy, yaitu sebesar Rp 200.000. Melalui operasi pengurangan, diperoleh nilai kembalian sebesar Rp 75.000 yang kemudian dipecah secara sistematis menggunakan kombinasi operasi pembagian integer dan modulo. Proses pemecahan dilakukan secara berurutan dari pecahan terbesar ke terkecil, menghasilkan 1 lembar Rp 50.000, 1 lembar Rp 20.000, dan 1 lembar Rp 5.000. Pecahan lainnya bernilai nol karena sisa kembalian sudah habis terbagi. Tidak terdapat sisa receh yang tidak terpecah, sehingga nilai donasi pada akhir struk tercatat Rp 0.

1.5 ANALISIS PROGRAM

```
#include <iostream>
```

Snippet “`#include <iostream>`” di atas merupakan *header file* standar dalam Bahasa C++ yang digunakan untuk menangani operasi masukan dan keluaran. *Header* ini memungkinkan penggunaan perintah seperti “`cin`” untuk *input* dan “`cout`” untuk *output* dalam program.

```
using namespace std;
```

Snippet “`using namespace std;`” di atas merupakan pernyataan yang memberitahu *compiler* bahwa program akan memanfaatkan seluruh fungsi dan objek yang terdapat dalam *namespace* `std`. Dengan adanya pernyataan ini, penulisan kode menjadi lebih ringkas karena tidak perlu menambahkan awalan “`std::`” setiap kali menggunakan elemen dari *namespace* tersebut.

```
int main(){
```

Snippet “`int main(){`” di atas merupakan fungsi utama yang wajib ada dalam setiap program C++. Kata kunci “`int`” menunjukkan bahwa fungsi ini akan mengembalikan nilai bertipe *integer* kepada sistem operasi setelah program selesai dijalankan. Seluruh logika dan instruksi program ditulis di dalam blok fungsi ini, yang ditandai dengan tanda kurung kurawal “`{`” sebagai pembuka dan “`}`” sebagai penutup.

```
int beras, telur, minyakGoreng;
```

Snippet “`int beras, telur, minyakGoreng;`” di atas merupakan deklarasi variabel yang menggunakan tipe data *integer*, sehingga program dapat menyimpan data masukan dari pengguna berupa bilangan bulat. Variabel-variabel tersebut nantinya akan digunakan untuk menampung jumlah atau kuantitas barang yang ada di Toko Sembako Jaya Abadi.

```
cout << "===== " << endl;
cout << "|          TOKO SEMBAKO JAYA ABADI          |" << endl;
cout << "===== " << endl << endl;
```

Snippet “`cout << "===== " << endl;`” di atas hingga seterusnya merupakan perintah untuk menampilkan output ke layar yang berfungsi sebagai *header* atau judul program. Karakter garis sama dengan (=) dan pipa (|) digunakan untuk membuat bingkai visual agar tampilan program “TOKO SEMBAKO JAYA ABADI” terlihat lebih rapi dan terstruktur bagi pengguna.

```
cout << "Harga Beras (Rp)          : "; cin >> beras;
cout << "Harga Telur (Rp)         : "; cin >> telur;
cout << "Harga Minyak Goreng (Rp) : "; cin >> minyakGoreng;
```

Snippet “cout << “Harga Beras (Rp) : ”; cin >> beras;” di atas hingga seterusnya merupakan kombinasi perintah *output* dan *input* yang digunakan secara bersamaan. Perintah “cout” berfungsi untuk menampilkan teks keterangan sebagai panduan bagi pengguna, kemudian “cin” digunakan untuk menerima nilai yang dimasukkan oleh pengguna melalui keyboard dan menyimpannya ke dalam variabel “beras”, “telur”, dan “minyakGoreng”. Ketiga baris ini dijalankan secara berurutan sehingga pengguna dapat memasukkan harga masing-masing barang satu per satu.

```
int totalBelanja = beras + telur + minyakGoreng;
    totalBelanja = totalBelanja + (totalBelanja * 25/100);
int uangRendy;
```

Snippet “int totalBelanja = beras + telur + minyakGoreng;” di atas merupakan deklarasi sekaligus inisialisasi variabel “totalBelanja” bertipe *integer*. Variabel ini dihitung menggunakan operasi penjumlahan dari ketiga variabel yang telah diinput sebelumnya, yaitu “beras”, “telur”, dan “minyakGoreng”. Kemudian “totalBelanja = totalBelanja + (totalBelanja * 25/100);” merupakan perhitungan PPN sebesar 25% yang langsung ditambahkan ke dalam variable “totalBelanja”. Nilai PPN diperoleh melalui operasi perkalian dan pembagian integer, kemudian hasilnya dijumlahkan dengan total belanja awal sehingga variable “totalBelanja” kini menyimpan nilai total belanja setelah pajak. Lalu, “int uangRendy;” merupakan deklarasi variable “uangRendy” bertipe *integer* yang akan digunakan untuk menyimpan nominal uang yang dibawa oleh Rendy sebagai *input* dari pengguna pada tahap berikutnya.

```
cout << "Total Belanja (+ PPN 25%) : " << totalBelanja << endl << endl;
```

Snippet “cout << “Total Belanja (+ PPN 25%) : ” << totalBelanja << endl << endl;” di atas merupakan perintah *output* yang digunakan untuk menampilkan hasil perhitungan total belanja setelah dikenakan PPN 25% kepada pengguna. Perintah “cout” mencetak teks keterangan “Total Belanja (+ PPN 25%) :” yang kemudian diikuti dengan nilai variable “totalBelanja” menggunakan operator “<<”. Adapun “endl” yang ditulis dua kali berfungsi untuk memberikan jarak dua baris setelah *output* ditambihkan, sehingga tampilan program menjadi lebih rapi dan mudah dibaca.

```
cout << "[Fathan]: Ren, totalnya segitu. Uang lu berapa?" << endl;
cout << "[Rendy] : Hmmmmmm, pajaknya gede juga ya." << endl;
cout << "          Nih than, my uang gweh segini" << endl << endl;
cout << "Nominal Uang Rendy (Rp) : "; cin >> uangRendy;
cout<< endl;
```

Snippet “cout << “[Fathan]: Ren, totalnya segitu. Uang lu berapa?” << endl;”, “cout << “[Rendy] : Hmmmmmm, pajaknya gede juga ya.” << endl;” dan “cout << “ Nih than, my uang gweh segini” << endl << endl;” di atas merupakan perintah *output* yang digunakan untuk menampilkan dialog antara karakter Fathan dan Rendy sebagai bagian dari tampilan antarmuka program agar terasa lebih interaktif, namun tidak mempengaruhi proses perhitungan. Kemudian, “cout << “Nominal Uang Rendy (Rp) : "; cin >> uangRendy;” merupakan kombinasi perintah *output* dan *input* yang digunakan untuk menerima nilai nominal uang dari pengguna dan menyimpannya ke dalam variable “uangRendy”. Sementara “cout<< endl;” hanya digunakan untuk baris kosong pemisah antar bagian *output* program.

```
int uangKembalian = uangRendy - totalBelanja;
```

Snippet “int uangKembalian = uangRendy - totalBelanja;” di atas merupakan deklarasi sekaligus inisialisasi variable “uangKembalian” bertipe *integer*. Nilai kembalian diperoleh melalui operasi pengurangan antara variable “uangRendy” dengan “totalBelanja”, sehingga program dapat mengetahui jumlah uang yang harus dikembalikan kepada pembeli.

```
cout << "===== " << endl;
cout << "|          STRUK KEMBALIAN          |" << endl;
cout << "===== " << endl;

cout << "[Bina]: Nih, than. Rincian kembaliannya: " << endl;
cout << "Total Kembalian: Rp " << uangKembalian << endl;
```

Snippet “cout << “===== ” << endl;” di atas hingga seterusnya merupakan perintah *output* yang digunakan untuk menampilkan *header* struk kembalian. Tampilan ini berfungsi untuk memberikan informasi kepada pengguna bahwa program telah memasuki tahap penampilan rincian kembalian. Kemudian, “cout << “[Bina]: Nih, than. Rincian kembaliannya: ” << endl; ” dan “cout << “Total Kembalian: Rp ” << uangKembalian << endl;” merupakan perintah *output* untuk menampilkan dialog diikuti dengan penampilan nilai total kembalian yang tersimpan dalam variable “uangKembalian”. Kedua baris tersebut berfungsi sebagai pembuka struk kembalian sebelum program memecah nilai kembalian ke dalam pecahan uang.

```

cout << "-----" << endl;
int lembarKoin = uangKembalian/100000
    uangKembalian = uangKembalian % 100000;
cout << "- Rp 100.000 : " << lembarKoin << " lembar" << endl;
lembarKoin = uangKembalian/50000;
uangKembalian = uangKembalian % 50000;
cout << "- Rp 50.000 : " << lembarKoin << " lembar" << endl;
lembarKoin = uangKembalian/20000;
uangKembalian = uangKembalian % 20000;
cout << "- Rp 20.000 : " << lembarKoin << " lembar" << endl;
lembarKoin = uangKembalian/10000;
uangKembalian = uangKembalian % 10000;
cout << "- Rp 10.000 : " << lembarKoin << " lembar" << endl;
lembarKoin = uangKembalian/5000;
uangKembalian = uangKembalian % 5000;
cout << "- Rp 5.000 : " << lembarKoin << " lembar" << endl;
lembarKoin = uangKembalian/1000;
uangKembalian = uangKembalian % 1000;
cout << "- Rp 1.000 : " << lembarKoin << " lembar" << endl;
lembarKoin = uangKembalian/1000;
uangKembalian = uangKembalian % 1000;
cout << "- Rp 1.000 : " << lembarKoin << " koin" << endl;
lembarKoin = uangKembalian/500;
uangKembalian = uangKembalian % 500;
cout << "- Rp 500 : " << lembarKoin << " koin" << endl;
lembarKoin = uangKembalian/200;
uangKembalian = uangKembalian % 200;
cout << "- Rp 200 : " << lembarKoin << " koin" << endl;
lembarKoin = uangKembalian/100;
uangKembalian = uangKembalian % 100;
cout << "- Rp 100 : " << lembarKoin << " koin" << endl;

```

Snippet “int lembarKoin = uangKembalian/100000, uangKembalian = uangKembalian % 100000;” di atas hingga seterusnya merupakan deklarasi sekaligus inisialisasi variable “lembarKoin” bertipe data *integer* yang digunakan secara berulang untuk menyimpan hasil pembagian *integer* “/” yang menunjukkan jumlah lembar atau koin dari setiap pecahan. variabel “lembarKoin” tidak dideklarasikan ulang pada pecahan berikutnya karena sudah dideklarasikan di awal. Sementara itu, operasi modulo “%” diterapkan pada variable “uangKembalian” di setiap tahap untuk memperoleh sisa variabel lembarKoin tidak dideklarasikan ulang pada pecahan berikutnya karena sudah dideklarasikan di awal nilai yang belum terpecah, yang kemudian diproses ke pecahan berikutnya hingga seluruh nilai kembalian habis terbagi.

```
cout << "-----" << endl;
cout << "[Fathan]: Aman! Sisa receh tak terpecahnya: Rp " << uangKembalian
    <<endl;
cout << "          Ini mau di Donasikan ga Ren? " << endl;
cout << "[Rendy] : GAS" << endl;
cout << "===== " << endl;
```

Snippet “cout << “[Fathan]: Aman! Sisa receh tak terpecahnya: Rp ” << uangKembalian <<endl;” di atas hingga seterusnya merupakan bagian penutup program yang menampilkan sisa receh yang tidak terpecah melalui variable “uangKembalian” beserta dialog antara karakter Fathan dan Rendy sebagai bagian dari antarmuka program. Nilai “uangKembalian” yang ditampilkan pada baris ini merupakan sisa akhir setelah seluruh proses pemecahan kembalian selesai dilakukan.

```
return 0;
```

Snippet “return 0;” di atas merupakan perintah yang digunakan untuk mengakhiri fungsi “main()” dengan mengembalikan nilai 0 kepada sistem operasi. Nilai 0 menandakan bahwa program telah berjalan dengan sukses tanpa adanya kesalahan.

1.6 KESIMPULAN

Berdasarkan hasil praktikum pada Modul I tentang Pemrograman Dasar, dapat diambil Kesimpulan bahwa:

1. Algoritma adalah sebuah metode yang digunakan untuk menyelesaikan sebuah permasalahan dengan langkah-langkah yang sistematis guna mendapatkan sebuah hasil yang lebih baik. *Pseudocode* merupakan sebuah deskripsi algoritma dengan struktur yang lebih sederhana dari beberapa bahasa pemrograman agar lebih mudah dimengerti oleh manusia. Sementara itu, *flow chart* merupakan suatu bagan dengan simbol-simbol khusus yang bertujuan untuk memberikan gambaran terkait urutan proses dan hubungan antara suatu proses (intruksi) dengan proses lainnya dalam suatu algoritma.
2. Dasar-dasar penulisan program komputer dapat dipahami melalui penerapan konsep *input*, proses, dan *output* sebagai alur utama sebuah program. Dalam praktikum ini, konsep tersebut diimplementasikan menggunakan perintah *cin* untuk menerima masukan dari pengguna, operasi aritmatika untuk memproses data, serta perintah *cout* untuk menampilkan hasil keluaran kepada pengguna. Pemahaman terhadap dasar-dasar ini menjadi fondasi penting dalam penulisan program yang terstruktur dan mudah dipahami.
3. Bahasa pemrograman C++ memiliki struktur yang terdiri dari dua bagian utama, yaitu *header* dan *body*. *Header* memuat *preprocessor directive* dan deklarasi yang diperlukan program, sementara *body* memuat seluruh logika dan instruksi yang akan dijalankan. Pemahaman terhadap struktur ini sangat penting dalam penulisan program yang teratur dan terstruktur.
4. Tipe data primitif digunakan untuk menentukan jenis nilai yang dapat disimpan dalam sebuah variabel. Beberapa tipe data dasar yang umum digunakan antara lain bilangan bulat (*int*) untuk nilai tanpa desimal, bilangan riil (*float* atau *double*) untuk nilai dengan desimal, karakter (*char*) untuk satuan huruf atau simbol, dan logika (*bool*) untuk nilai benar atau salah. Pemilihan tipe data yang tepat sangat berpengaruh terhadap keakuratan hasil perhitungan program.
5. *Typecasting* adalah proses konversi dari satu jenis tipe data ke tipe data lainnya, baik secara eksplisit oleh *programmer* maupun secara implisit oleh *compiler*. Hal ini penting dilakukan untuk menghindari kehilangan nilai atau kesalahan komputasi ketika dua variabel dengan tipe data berbeda digunakan dalam satu operasi aritmatika.

DAFTAR PUSTAKA

- [1] N. Chairunnisa, *Langkah Mudah Belajar Pemrograman C++ Untuk Pemula*. Yogyakarta: Anak Hebat Indonesia, 2024. R. Munir, *Pemrograman Bahasa C dan Implementasinya*, Bandung, Indonesia: Informatika, 2011.
- [2] Falahah, *Algoritma & Pemrograman: Implementasi dengan Python pada Google Colab*. Bandung: Remaja Rosdakarya, 2023.
- [3] A. B. Kaswar, Nurjannah, dan S. G. Zain, *Mudah Belajar Pemrograman Dasar C++*. Banda Aceh: Syiah Kuala University Press, 2021.
- [4] M. Nugraha, *Dasar Pemrograman dengan C++: Materi Paling Dasar Untuk Menjadi Programmer Berbagai Platform*. Yogyakarta: Deepublish, 2021.
- [5] V. Alliana and Y. Yahfizham, "Analisis Penerapan Dan Fungsi Algoritma Pemrograman Pada Komputer," *Jurnal Riset Rumpun Matematika dan Ilmu Pengetahuan Alam (JURRIMIPA)*, vol. 3, no. 1, pp. 207-218, Apr. 2024.